Structural Cryptographic Engine (SCE) Specification v1.0

Joseph Forrester

December 2025

Contents

1 Introduction

The **Structural Cryptographic Engine** (SCE) is a novel cryptographic construction in which encryption security is derived from the controlled evolution of a *structural dynamical system*. Unlike traditional block ciphers, which operate on fixed algebraic transformations, SCE derives keystream and authentication artifacts from the behavior of a well-defined structural engine (Section 4) whose state evolves under nonlinear, coupled, negative-space dynamics.

SCE has three primary components:

1. The **SCE Engine** — a canonical structural dynamical system defined in $\texttt{sce}_e ngine.tex. This engine is the man$

1. `The SCE-DRBG - a Deterministic Random Bit Generator built by repeatedly stepping the structural engine and extracting its mosaic signatures.`
2. `The SCE-AEAD mode - a misuse-resistant authenticated encryption construction using a Synthetic IV (SCE-SIV) tied to the plaintext structure.`

All cryptographic behavior in SCE ultimately depends on the engine's nonlinear structural updates, negative-space folding, and mosaic extraction rules. SCE does **not** depend on any other framework, runtime, or external architecture. It is a standalone cryptosystem.

1.1 Intended Use Cases

SCE is designed for applications requiring:

- encryption where the structure of the underlying data or model carries meaning,
- cryptographic protection that remains nontrivial to interpret even after ciphertext compromise,
- resilience to nonce reuse via SIV semantics,
- high-entropy keystreams from deterministic seeds,
- auditability and replay-deterrence,
- and integration into systems where additional semantic security is desired beyond conventional byte-level ciphers.

SCE is suitable for:

- protecting structured data artifacts,
- AI/ML model states,
- proprietary structural representations,
- high-assurance embedded systems,
- or any workload where data structure, absence, and contrast are meaningful security dimensions.

1.2 Process Flow Overview

High-level operation:

1. A master key, associated data, nonce, and plaintext are supplied.
2. A synthetic IV (SIV) is computed from these components.
3. Key material for engine seeding and keystream extraction is derived from the SIV using HKDF.
4. The canonical structural engine is initialized deterministically from the derived seed.

5. The engine is stepped repeatedly, producing mosaic vectors that are compressed into keystream blocks.
6. The keystream XORs with plaintext to produce ciphertext.
7. A MAC over $(AD, \text{nonce}, C, \text{SIV})$ yields the tag.

Every one of these steps depends directly on the engine defined in Section 4. No other engines are permitted in SCE v1.0.

2 Definitions and Abbreviations

All terms here are normative for SCE v1.0.

- **SCE**: Structural Cryptographic Engine.
- **SCE Engine**: The canonical dynamical system defined in Section 4. This engine is mandatory and normative.
- **DRBG**: Deterministic Random Bit Generator — a keyed, deterministic function that produces pseudorandom bytes.
- **SCE-DRBG**: The DRBG built by stepping the canonical SCE Engine.
- **AEAD**: Authenticated Encryption with Associated Data.
- **SCE-SIV**: Synthetic IV derived from $(AD, \text{nonce}, H(P))$ using BLAKE2s.

3 Notation

Cryptographic primitives:

- HKDF: HMAC-based key derivation (HMAC-SHA-256).
- BLAKE2s$_k$: keyed BLAKE2s MAC.
- $\oplus$: XOR on byte strings.

Structural engine quantities:

- X_t: signal matrix at time t.
- Y_t: negative-space matrix at time t.
- M_t: mosaic signature extracted at time t.
- $S_t = (X_t, Y_t)$: total engine state.

All byte strings MUST be length-aware in implementation. All floats MUST be deterministically mapped to bytes for KATs.

4 Canonical Structural Engine v1.0

4.1 Purpose and Design Goals

This section defines the **canonical structural engine** used by SCE v1.0. The engine is the source of structural entropy for the SCE-DRBG and therefore for the SCE AEAD mode.

The engine is designed to:

- Be fully deterministic and seedable from cryptographic material.
- Operate on a bounded, continuous state space.
- Exploit *negative-space structure*: each state is paired with a complementary "dual" state, and the engine dynamically mixes both.
- Exhibit sensitive dependence on the seed, while remaining numerically stable for fixed parameters.
- Produce high-dimensional mosaic signatures that are compressed into keystream blocks.

This canonical engine is an evolution of earlier Fractal–Mosaic (FMM) prototypes and subsequent FMI / "Spider-Eyes" / negative-space formulations. It is written here as a self-contained, mathematically explicit design; earlier prototypes should be treated as historical motivation, not normative definitions.

4.2 State Space and Parameters

The structural engine maintains a collection of coupled *gates*, each gate consisting of a *signal* and a *negative-space* component.

4.2.1 State Space

Let:

- G be the number of gates (e.g., $G = 8$).
- d be the dimensionality of each gate vector (e.g., $d = 4$).

The engine state at time step t is:

$$S_t = (X_t, Y_t)$$

where:

- $X_t \in [-1, 1]^{G \times d}$ is the *signal matrix*.
- $Y_t \in [-1, 1]^{G \times d}$ is the *negative-space matrix*.

Intuitively:

- X_t represents "what is present" structurally.
- Y_t represents "what is absent" or the complementary space.

The engine also maintains a set of fixed parameter matrices derived from the seed:

$$\Theta = \{W^{(x)}, W^{(y)}, B^{(x)}, B^{(y)}, C^{(xy)}, C^{(yx)}\}$$

with:

- $W^{(x)}, W^{(y)} \in \mathbb{R}^{G \times G}$: gate-to-gate coupling weights for X and Y.
- $B^{(x)}, B^{(y)} \in \mathbb{R}^{G \times d}$: per-gate biases.

- $C^{(xy)}, C^{(yx)} \in \mathbb{R}^{G \times G}$: cross-coupling matrices mixing X into Y and Y into X.

All of these matrices are deterministically derived from the engine seed and remain constant for the lifetime of a seeded engine instance.

4.2.2 Recommended Parameter Set (Normative for v1.0)

For SCE v1.0, the following parameter values are RECOMMENDED:

- $G = 8$ gates.
- $d = 4$ dimensions per gate.
- State bounds: $X_t, Y_t \in [-1, 1]^{G \times d}$ enforced by a clipping operator (Section 4.4).

Other choices MAY be explored in research, but interoperability and KATs for SCE v1.0 assume this configuration.

4.3 Seeding and Initialization

The engine must be seeded deterministically from cryptographic material. The goal is:

- Small changes in the seed produce large, unpredictable changes in long-run trajectories.
- The mapping from seed to (S_0, Θ) is deterministic and reproducible across implementations.

4.3.1 Seed Derivation from SCE Context

Given SCE-DRBG inputs $(k_{\text{enc}}, \text{nonce}, AD, \text{SIV})$, the engine seed is derived using HKDF:

$$\text{prk}_{\text{eng}} = \text{HKDF\_Extract}(\text{salt} = \text{“SCE-ENGINE-SEED”}, k_{\text{enc}})$$

$$\text{seed} = \text{HKDF\_Expand}(\text{prk}_{\text{eng}}, \text{“SCE-ENGINE-CTX”} \parallel \text{nonce} \parallel AD \parallel \text{SIV}, \ell_{\text{seed}})$$

where ℓ_{seed} is a fixed seed length (e.g., 256 bytes).

4.3.2 Seed-to-Parameter Mapping

The seed is consumed by a simple, deterministic pseudorandom generator (e.g., SplitMix64 or a fixed XOF) to construct:

- Initial state $S_0 = (X_0, Y_0)$.
- Parameter set Θ.

A concrete instantiation:

1. Interpret seed as input to a fixed XOF (e.g., BLAKE2s in XOF mode) to generate a stream of bytes.
2. Map contiguous chunks of bytes to floating-point values in $[-1, 1]$ by:

$$z = \frac{\text{int\_32\_from\_bytes}}{2^{31}} - 1.$$

3. Fill X_0, Y_0, $W^{(x)}$, $W^{(y)}$, $B^{(x)}$, $B^{(y)}$, $C^{(xy)}$, and $C^{(yx)}$ in a fixed order from this stream.

Implementations MUST define the integer-to-float mapping and filling order in a KAT-compatible way.

4.3.3 Negative-Space Initialization

To ensure a meaningful negative-space relation at $t = 0$, we impose:

$$Y_0 = -X_0$$

after X_0 has been filled. This creates an initial signal/anti-signal relationship. Subsequent dynamics are free to deviate from perfect antisymmetry due to cross-coupling and nonlinearity.

4.4 Update Rule

The engine evolves in discrete time steps. For each step, it updates X_t and Y_t using coupled nonlinear transformations and then applies a bounding operator to keep the state within $[-1, 1]$.

4.4.1 Nonlinear Activation

Define a smooth, odd activation function:

$$\phi(z) = \tanh(\alpha z),$$

where $\alpha > 0$ controls nonlinearity (e.g., $\alpha = 1.5$). This function is applied element-wise to matrices.

4.4.2 Gate Coupling

Let:

$$\tilde{X}_t = W^{(x)} X_t + C^{(xy)} Y_t + B^{(x)},$$

$$\tilde{Y}_t = W^{(y)} Y_t + C^{(yx)} X_t + B^{(y)}.$$

Here:

- $W^{(x)} X_t$ applies gate-to-gate interactions in the signal space.
- $C^{(xy)} Y_t$ injects negative-space structure into X.
- $W^{(y)} Y_t$ and $C^{(yx)} X_t$ analogously update Y.
- $B^{(x)}, B^{(y)}$ provide bias and offset.

All matrix multiplications are over the gate dimension; biases broadcast over the d dimensions.

4.4.3 Negative-Space Folding

To preserve a relationship between X and Y and to capture the negative-space intuition, we define a folding operator:

$$F(X_t, Y_t) = \beta(X_t - Y_t),$$

$$G(X_t, Y_t) = \beta(Y_t - X_t),$$

where $\beta \in (0, 1]$ controls the strength of the folding (e.g., $\beta = 0.5$). This enforces a tendency for X and Y to remain anti-correlated while still allowing dynamics to explore the state space.

4.4.4 Update Equations

The full update equations for one time step are:

$$X_{t+1} = \text{clip}_{[-1,1]} \left(\phi(\tilde{X}_t) + F(X_t, Y_t) \right),$$

$$Y_{t+1} = \text{clip}_{[-1,1]} \left(\phi(\tilde{Y}_t) + G(X_t, Y_t) \right),$$

where $\text{clip}_{[-1,1]}$ clamps each element of the matrix to the interval $[-1, 1]$.
This yields:

- Nonlinearity via ϕ.
- Cross-coupling between X and Y.
- Negative-space folding via F and G.
- Bounded trajectories for all t.

4.5 Mosaic Extraction

The engine must provide an observable signature M_t at each step that captures both local gate behavior and global structure. These mosaics are later compressed into keystream blocks.

4.5.1 Per-Gate Projection

For each gate $g \in \{1, \ldots, G\}$, define:

$$v_g^{(x)} = X_t[g, :], \quad v_g^{(y)} = Y_t[g, :].$$

Compute a gate-level projection:

$$m_g = \begin{bmatrix} v_g^{(x)} \\ v_g^{(y)} \\ v_g^{(x)} - v_g^{(y)} \\ v_g^{(x)} \odot v_g^{(y)} \end{bmatrix} \in \mathbb{R}^{4d},$$

where $\odot$ denotes element-wise multiplication. This captures:

- Signal content.
- Negative-space content.
- Their difference (contrast).
- Their interaction (co-activation).

4.5.2 Global Mosaic Vector

The full mosaic vector is:

$$M_t = \text{concat}(m_1, m_2, \ldots, m_G) \in \mathbb{R}^{4dG}.$$

For the recommended parameters $G = 8$, $d = 4$, M_t has dimension $4 \cdot 4 \cdot 8 = 128$.

4.5.3 Quantization

For cryptographic use, M_t is quantized to bytes. A simple scheme:

1. Map each real component $z \in [-1, 1]$ to an integer:

$$q = \left\lfloor \frac{(z+1)}{2} \cdot 255 \right\rfloor \in \{0, \ldots, 255\}.$$

2. Pack the resulting integers into a 128-byte vector.

Quantization MUST be deterministic and specified in KATs.

4.6 Keystream Block Formation

SCE-DRBG converts sequences of mosaics into keystream blocks using a keyed hash.

4.6.1 Block Aggregation

For each block i, aggregate T successive mosaics:

$$\mathcal{M}_i = (M_{t_i}, M_{t_i+1}, \ldots, M_{t_i+T-1}),$$

and define:

$$B_i = \text{BLAKE2s}_{k_{\text{blk}}}(\text{“SCE-BLOCK”} \parallel i \parallel \mathcal{M}_i),$$

where:

- k_{blk} is a subkey derived from k_{enc}.
- i is the block index encoded as a fixed-length integer.

For SCE v1.0, $T = 4$ mosaics per block and B_i is 32 bytes.

4.6.2 Keystream Concatenation

Blocks B_i are concatenated until the requested keystream length L is reached, and the result is truncated to L bytes.

$$KS = B_0 \parallel B_1 \parallel \ldots \quad \text{(truncated to } L\text{)}.$$

4.7 Informal Rationale (Non-normative)

This subsection is *informative* and explains the design choices.

- **Negative-space pairing** (X_t, Y_t) encodes both what is present and what is structurally “missing”. The folding terms F, G bind them together so that trajectories are sensitive to signal/anti-signal relationships.
- **Nonlinear coupling** through ϕ and the matrices $W^{(\cdot)}, C^{(\cdot)}$ introduces high-dimensional, nontrivial dynamics.
- **Mosaic extraction** uses both first-order and mixed terms, making it harder for an attacker to invert the mapping from B_i back to internal state without knowing all parameters.
- **Hash-based compression** (BLAKE2s with a subkey) bends structural complexity into a fixed-size pseudorandom block, analogous to a sponge or extract-then-compress structure.

Cryptanalytic work is still required to evaluate:

- Whether the engine exhibits undesirable correlations in practice.
- Whether reduced-round variants leak structure through M_t.
- How resistant the full SCE-DRBG is to state-recovery or backtracking attacks under realistic adversarial models.

The engine defined here is a concrete, mathematically explicit starting point suitable for Known Answer Tests and experimental analysis. It is not presented as a final, proven construction, but as the canonical baseline for SCE v1.0.

5 SCE-DRBG (Deterministic Random Bit Generator)

The SCE-DRBG is built **exclusively** from the canonical structural engine described in Section 4. No alternative engines are permitted in SCE v1.0.

5.1 Inputs

- k_{enc}: encryption subkey derived from SCE-SIV.
- nonce: per-message nonce.
- AD: associated data.
- SIV: synthetic IV.
- L: desired keystream length in bytes.

5.2 Deterministic Seeding

The DRBG derives the engine seed using HKDF:

$$\text{prk}_{\text{eng}} = \text{HKDF\_Extract}(\text{“SCE-ENGINE-SEED”}, k_{\text{enc}})$$

$$\text{seed} = \text{HKDF\_Expand}(\text{prk}_{\text{eng}}, \text{“SCE-ENGINE-CTX”} \parallel \text{nonce} \parallel AD \parallel \text{SIV}, \ell_{\text{seed}})$$

The seed maps deterministically to the engine's initial state S_0 and parameter set Θ.

5.3 Stepping the Engine

Let the engine's update rule be the normative equations defined in Section 4. The engine is stepped repeatedly:

$$S_{t+1} = f(S_t; \Theta)$$

For each step, a mosaic M_t is extracted exactly as in Section 4.5.

5.4 Block Compression

Mosaics are aggregated into blocks of T consecutive mosaics (e.g., $T = 4$), and compressed via keyed BLAKE2s:

$$B_i = \text{BLAKE2s}_{k_{\text{blk}}}(\text{“SCE-BLOCK”} \parallel i \parallel M_{t_i} \parallel \cdots \parallel M_{t_i+T-1}),$$

where k_{blk} is derived deterministically from k_{enc}.

5.5 Keystream Output

Blocks B_i are concatenated and truncated to exactly L bytes. This keystream KS is the output of SCE-DRBG.

6 SCE-SIV and AEAD Construction

SCE uses a deterministic SIV-style AEAD mode. All keystreams come from the **canonical SCE Engine**. No alternative engines are allowed.

6.1 SCE-SIV Definition

Let $H(\cdot)$ be BLAKE2s-256. The synthetic IV is:

$$\text{SIV} = \text{BLAKE2s}_k(AD \parallel \text{nonce} \parallel H(P)).$$

SIV MUST be computed exactly this way.

6.2 Key Derivation

From the master key k and SIV:

$$\text{prk} = \text{HKDF\_Extract}(\text{“SCE-SIV-EXTRACT”}, k)$$

$$k_{\text{enc}} = \text{HKDF\_Expand}(\text{prk}, \text{“SCE-SIV-ENC”} \parallel \text{SIV}, 32)$$

All SCE engine seeding and DRBG state derive from k_{enc}.

6.3 Encryption

1. Compute SIV.
2. Derive k_{enc}.
3. Run SCE-DRBG to produce KS of length $|P|$.
4. Compute ciphertext:
$$C = P \oplus KS.$$
5. Compute authentication tag:
$$\text{Tag} = \text{BLAKE2s}_k(AD \parallel \text{nonce} \parallel C \parallel \text{SIV}).$$
6. Output $(C, \text{Tag}, \text{SIV})$.

6.4 Decryption

Given $(C, \text{Tag}, \text{SIV})$:

1. Derive k_{enc} from SIV.
2. Run SCE-DRBG to produce KS of length $|C|$.
3. Compute:
$$P' = C \oplus KS.$$
4. Recompute SIV’ from P'.
5. Recompute Tag’ from (C, SIV').
6. Accept P' iff $SIV' = SIV$ and $Tag' = Tag$.

7 Known Answer Tests (KATs)

KATs in SCE v1.0 MUST validate the **entire engine + DRBG + AEAD** pipeline. Partial tests are insufficient.

7.1 DRBG KAT Requirements

Each DRBG vector MUST contain:

- key k,
- nonce,
- associated data AD,
- SIV,
- output length L,
- engine seed (hex),
- first N mosaics $M_0, \ldots, M_{N-1}$,
- final keystream KS (hex).

DRBG output MUST match exactly, bit-for-bit.

7.2 AEAD KAT Requirements

Each AEAD vector MUST include:

- key k,
- nonce,
- associated data AD,
- plaintext P,
- ciphertext C,
- SIV,
- Tag.

Implementations MUST:

- Produce identical $(C, \text{SIV}, \text{Tag})$ for encryption.
- Recover P exactly during decryption.

8 Security Considerations

8.1 Novel Security Class

SCE defines a new class of cryptographic systems: **structural cryptography** — where security arises from the evolution of structured dynamical systems, not from fixed algebraic permutations.

8.2 Engine Dependency

All SCE security depends directly on the engine defined in Section 4. No alternative engines are permitted in SCE v1.0. If the engine is altered, the system is no longer SCE.

8.3 Misuse Resistance

The SCE-SIV construction mitigates catastrophic nonce reuse: re-encrypting with the same nonce does not trivially expose input structure. However, SCE does not encourage deliberate nonce reuse.

8.4 Structural Bias Risk

Because SCE relies on nonlinear structural dynamics, the DRBG may leak statistical bias if the engine is improperly parameterized. Implementers MUST test DRBG outputs using NIST STS, Dieharder, or equivalent suites.

8.5 Side-Channel Risks

Implementers MUST consider:

- constant-time constraints,
- cache and timing side channels,
- floating-point determinism,
- matrix multiplication leakage.

Failure to implement these protections may leak internal engine state.

9 Appendix

9.1 Reference to FMM–CF

The original FMM-based cryptographic prototype is documented in the "Fractal–Mosaic Cryptographic Framework (FMM–CF)" Appendix A (2025). That document includes:

- A concrete FMM-DRBG design.
- An FMM-based SIV-style AEAD construction.
- Hedged mode using ChaCha20.
- Example implementation notes and preliminary diagnostic results.

SCE generalizes this pattern to allow any structural engine that implements the SCE-DRBG interface.

9.2 Engine Specifications (Out of Scope)

The detailed mathematics of specific engines (FMM, FMI, Spider-Eyes, Not Math, etc.) are out of scope for this core SCE specification and SHOULD be defined in separate documents:

- *FMM-Engine-Spec v1.x*
- *FMI-Engine-Spec v1.x*
- *SpiderEyes-Engine-Spec v1.x*

Each engine document MUST:

- Define the state space S_t.
- Define initialization from a seed.
- Define the update rule $S_{t+1} = f(S_t)$.
- Define the construction of mosaic signatures M_t.
- Specify how M_t are packed into keystream blocks.

9.3 Recommended Parameters (Initial)

For initial experimentation:

- Key size: 256 bits.
- Nonce size: 96 bits (12 bytes).
- Hash: BLAKE2